Python Activity-Based Learning: Case Studies 1 to 10

Repository: Python-Activity-Based-Learning | Syllabus: Computational Thinking, Algorithms, Flowcharts, Pseudocode, Python Solutions, Test Cases

Case Study 1: University Scholarship Allocation

Problem: Evaluate applicants using aggregate marks, subject minimums, attendance, and income criteria with tie-breaking. Algorithm: 1. Input student records. 2. Check eligibility (marks >= 375, attendance >= 80%, income <= 5L). 3. Sort by marks DESC, attendance DESC, income ASC. 4. Output ranked recipients.

Python Code:

```
def allocate_scholarships(applicants):
    eligible = []
    for app in applicants:
        name, marks, s1, s2, s3, att, inc = app
        if marks >= 375 and s1 >= 60 and s2 >= 60 and s3 >= 60 and att >= 80 and inc <= 500000:
            eligible.append((name, marks, att, inc))
    eligible.sort(key=lambda x: (-x[1], -x[2], x[3]))
    return eligible
```

Case Study 2: Progressive Electricity Billing

Problem: Calculate household bills using consumption slabs, fixed charges, and surcharges; detect unusual spikes. Algorithm: 1. Input monthly units. 2. Calculate slab-wise energy charges. 3. Add fixed charges and surcharges. 4. Compare consecutive months and flag spikes > 30%.

Python Code:

```
def calculate_electricity_bill(monthly_units):
    bills = []
    prev_bill = 0
    for i, units in enumerate(monthly_units):
        energy = units * 3.50 if units <= 100 else (350 + (units-100)*5.25 if units <= 300 else 1400 + (units-300)*7.8
        total = energy + 150 + (energy * 0.05)
        spike = True if i > 0 and total > prev_bill * 1.30 else False
        bills.append((units, total, spike))
        prev_bill = total
    return bills
```

Case Study 3: Hospital Billing and Patient Classification

Problem: Process hospital charges, apply senior/insurance concessions, and classify patient categories. Algorithm: 1. Input consultation, diagnostics, room charges, and age. 2. Compute gross total and apply 15% concession if age >= 60. 3. Deduct insurance and determine net payable.

Python Code:

```
def process_hospital_bill(name, age, consultation, diagnostics, room, has_insurance):
    gross = consultation + diagnostics + room
    concession = 0.15 * gross if age >= 60 else 0
    taxable = gross - concession
    insurance_paid = 0.8 * taxable if has_insurance else 0
    net_payable = taxable - insurance_paid
    return name, gross, concession, insurance_paid, net_payable
```

Case Study 4: Employee Payroll and Deduction Analysis

Problem: Compute gross salary from basic pay, DA, HRA, overtime; deduct PF, professional tax, and tax slabs. Algorithm: 1. Input basic salary and overtime hours. 2. Compute allowances (DA=40%, HRA=20%). 3. Calculate gross, apply deductions, and output net salary.

Python Code:

```
def calculate_payroll(basic, overtime_hrs):
```

```
    da = basic * 0.40
    hra = basic * 0.20
    overtime = overtime_hrs * 250
    gross = basic + da + hra + overtime
    pf = basic * 0.12
    pt = 200
    net = gross - (pf + pt)
    return gross, pf, net
```

Case Study 5: Multi-Plan Telecommunications Cost Optimization

Problem: Evaluate mobile plans against voice and data requirements; calculate overages and find the cheapest plan. Algorithm: 1. Input required voice and data. 2. Loop through plan catalog. 3. Filter eligible plans and calculate total cost with overage fees. 4. Select minimum cost plan.

Python Code:

```
def optimize_telecom_plan(plans, req_voice, req_data):
    eligible = []
    for name, base_cost, limit_voice, limit_data, rate_v, rate_d in plans:
        over_v = max(0, req_voice - limit_voice) * rate_v
        over_d = max(0, req_data - limit_data) * rate_d
        total = base_cost + over_v + over_d
        eligible.append((name, total))
    return min(eligible, key=lambda x: x[1])
```

Case Study 6: Railway Fare and Reservation Calculator

Problem: Calculate railway fares using distance slabs, travel classes, age concessions, and reservation fees. Algorithm: 1. Input distance, class, and age. 2. Compute base fare per km based on class multiplier. 3. Apply age concession and add reservation fee.

Python Code:

```
def calculate_railway_fare(distance, travel_class, age):
    rate = 1.5 if travel_class == 'Sleeper' else (2.5 if travel_class == 'AC3' else 3.5)
    base = distance * rate
    concession = 0.5 * base if age >= 60 else 0
    return max(50, base - concession + 40)
```

Case Study 7: Retail Billing with Multiple Discount Rules

Problem: Process purchased items, category taxes, and threshold volume discounts to generate an itemized invoice. Algorithm: 1. Input cart items. 2. Compute item totals with category GST. 3. Apply 10% threshold discount if total > ■5,000.

Python Code:

```
def generate_retail_invoice(cart):
    subtotal = sum(price * qty for _, price, qty, _ in cart)
    discount = 0.10 * subtotal if subtotal > 5000 else 0
    final_total = subtotal - discount
    return subtotal, discount, final_total
```

Case Study 8: Manufacturing Cost and Profitability Analysis

Problem: Compute material costs, labor, wastage, and machine depreciation to check production profitability targets. Algorithm: 1. Input material cost, wastage %, labor hours, and selling price. 2. Compute total manufacturing cost. 3. Compare against target profit margin.

Python Code:

```
def analyze_manufacturing(material_cost, wastage_pct, labor_hrs, wage, selling_price):
    mat_adjusted = material_cost * (1 + wastage_pct / 100)
    total_cost = mat_adjusted + (labor_hrs * wage)
    profit = selling_price - total_cost
    return total_cost, profit, profit >= 0.20 * total_cost
```

Case Study 9: Multi-Currency Travel Budget Analysis

Problem: Convert travel expenses using exchange rates, add bank transaction fees, and identify peak expense category. Algorithm: 1. Input expenses in foreign currencies. 2. Convert using exchange rate dictionary and add 2% fee. 3. Find maximum expense category.

Python Code:

```
def analyze_travel_budget(expenses, rates):
    converted = {cat: amt * rates[curr] * 1.02 for cat, (amt, curr) in expenses.items()}
    total = sum(converted.values())
    peak = max(converted, key=converted.get)
    return total, peak
```

Case Study 10: Campus Event Financial Planning

Problem: Aggregate event costs, subtract sponsorships, and compute break-even registration counts. Algorithm: 1. Input fixed costs and sponsorships. 2. Compute net deficit. 3. Calculate break-even participant count.

Python Code:

```
def calculate_event_breakeven(fixed_costs, sponsorship, fee_per_head, var_cost_per_head):
    net_fixed = sum(fixed_costs.values()) - sponsorship
    contribution = fee_per_head - var_cost_per_head
    breakeven_count = net_fixed / contribution if contribution > 0 else float('inf')
    return net_fixed, int(breakeven_count) + 1
```